Simple Calculator

课程项目汇报

授课教师：于仕琪老师

年级专业：2023级计算机科学与技术

学　　号：12310907

姓　　名：吴嘉淇

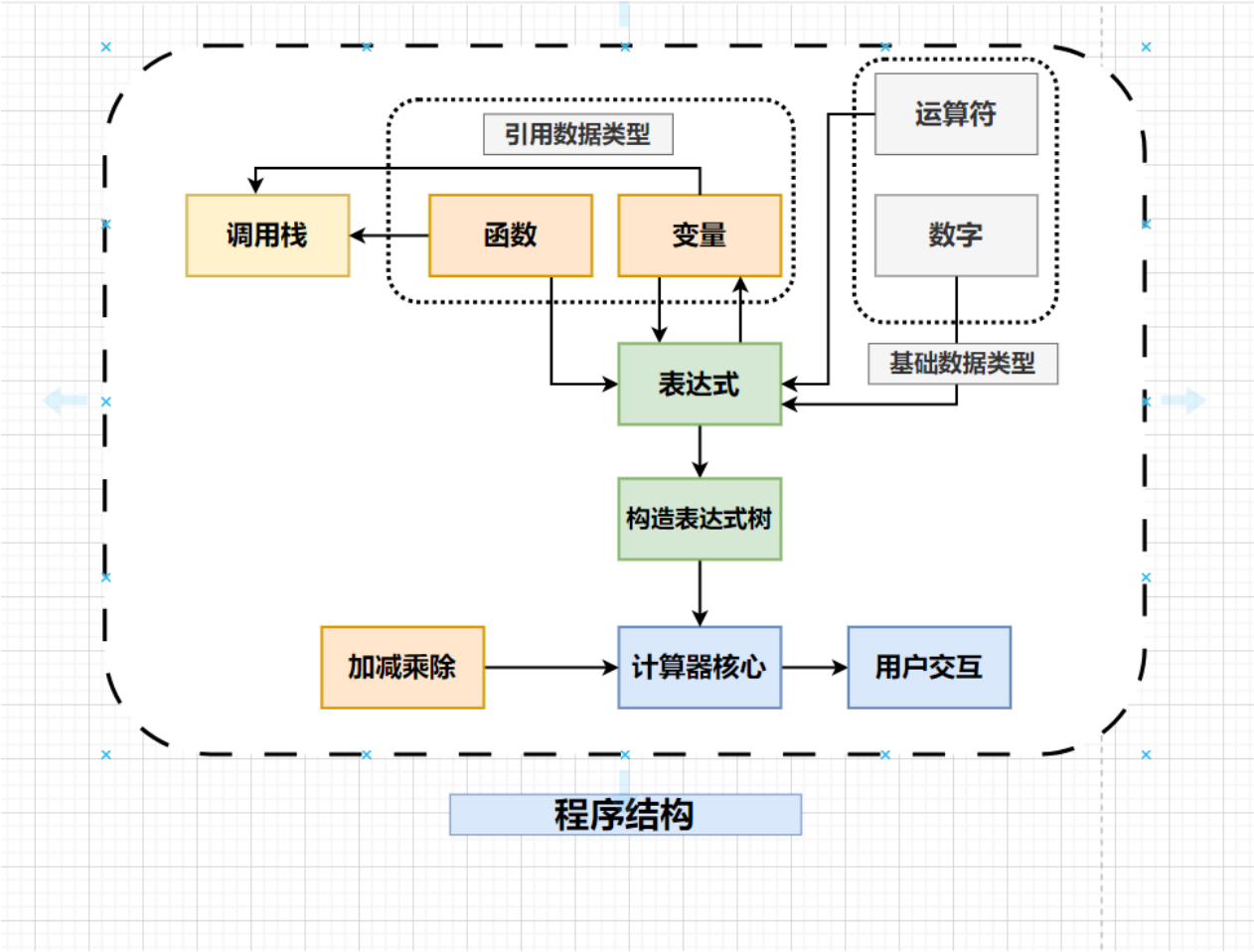
目录

摘要

- **Simple Calculator**
  - **课程项目汇报**
- **目录**
  - 0. 摘要
  - 1. 主要功能演示
    - 1.1 简要需求分析
    - 1.2 基础运算功能
      - 1.2.1 基本四则运算
      - 1.2.2 复杂表达式计算
      - 1.2.3 高精度计算
    - 1.3 变量定义与使用
      - 1.3.1 变量定义与赋值
      - 1.3.2 变量表显示
      - 1.3.3 变量更新
    - 1.4 函数定义与调用
      - 1.4.1 函数定义
      - 1.4.2 函数调用
    - 1.5 错误处理机制
      - 1.5.1 语法错误处理
      - 1.5.2 运行时错误处理
      - 1.5.3 内存错误处理
  - 2. 核心难点分析
    - 2.1 高精度计算实现
  - 2.1.1 高精度加法
    - 2.1.1.1add_big_abs 方法
    - 2.1.1.2add_big_decimal 方法
    - 2.1.1.3complete_add 方法
  - 2.1.2 高精度除法
    - 2.2 表达式解析
    - 2.2.1 tokenize方法
      - 算法流程：
    - 2.2.2 calculate_core方法
      - 递归计算流程：
  - 3. 项目完成感言
  - 参考文献

0. 摘要

本项目是一个相对完整的基于命令行的简单计算器，通过表达式树结构实现了复杂表达式的解析与计算，并使用字符串模拟实现了高精度运算功能。除了基本的加减乘除运算，计算器还实现了变量定义和自定义函数声明的高级功能，我会在后续的报告里进行演示说明。



项目大致结构

1. 主要功能演示

在这部分中，我将首先展示本项目实现的一些功能的运行截图以及对于结果的说明。

1.1 简要需求分析

在本项目的实现中，我确定了以下的几个核心需求：

基本功能：

1. **高精度四则运算**：支持超出基本数据类型范围的大数计算
2. **双模式操作界面**：提供命令行模式和交互模式两种使用方式
3. **基础命令支持**：实现 `-h`（帮助）与 `-v`（版本信息）等基本指令(如果需要在名领行模式使用这些命令,请严格按照./calculator [数字1] [操作符] [数字2] [可选命令] 的结构输入!)

高级功能：

4. **变量系统**：支持变量的定义、使用和更新
5. **函数系统**：支持用户自定义函数及对函数的单独调用
6. **表达式计算**：支持复杂数学表达式的解析与运算

项目的实现过程将围绕上述需求展开，在保证基础功能正确性的同时，我通过使用一些在其它课程（dsaa/机组等）中学到的知识尝试实现了一些高级功能。

以下是对于各个功能的演示。

1.2 基础运算功能

计算器支持基本的四则运算和复杂表达式计算，特别是具备高精度计算的能力。

（除法在精度为20到50时与bc计算结果基本完全匹配，但是在部分情况可能会出现舍入误差，例如1/2会计算为0.49999...）

1.2.1 基本四则运算

以下是基本四则运算的示例：

```
wu@w5712:~/CppLearning/CS205/projectFinal1$ ./calculator
Simple Calculator (type 'quit' to exit)
> 123 + 456
Result: 579
> 123 - 456
Result: -333
> 123 x 456
Result: 56088
> 123 / 456
Result: 0.26973684210526315789
>
```

：计算器交互模式下的基本四则运算示例

```
问题    输出    调试控制台    终端    端口    评论

Goodbye!
wu@w5712:~/CppLearning/CS205/projectFinal1$ ./calculator 320918 x 178719.12390 / 23
Invalid arguments. Use --help for more information.
wu@w5712:~/CppLearning/CS205/projectFinal1$ ./calculator 320918 x 178719.12390
57354183803.7402
wu@w5712:~/CppLearning/CS205/projectFinal1$ ./calculator 320918 / 412
778.92718446601941747572
wu@w5712:~/CppLearning/CS205/projectFinal1$ ./calculator 320918 + 13241235123
13241556041
wu@w5712:~/CppLearning/CS205/projectFinal1$ ./calculator 320918 - 129378965892371
-129378965571453
wu@w5712:~/CppLearning/CS205/projectFinal1$
```

：计算器命令行模式下的基本四则运算示例(命令行模式不支持表达式运算)

1.2.2 复杂表达式计算

计算器的交互模式支持带括号的复杂表达式计算：

```
quit
wu@w5712:~/CppLearning/CS205/projectFinal1$ ./calculator
Simple Calculator (type 'quit' to exit)
> 777897120897129803 / 1740273878919823 + 71893749218 x 102380
Result: 7360482044939286.99695278996293989874
> 71829 / 710238 x 812903
Result: 82211.89177008270466486756
> quit
Goodbye!
wu@w5712:~/CppLearning/CS205/projectFinal1$ bc
```

：计算器交互模式下的复杂表达式运算示例

```
wu@w5712:~/CppLearning/CS205/projectFinal1$ bc
bc 1.07.1
Copyright 1991-1994, 1997, 1998, 2000, 2004, 2006, 2008, 2012-2017 Free Software Foundation, Inc.
This is free software with ABSOLUTELY NO WARRANTY.
For details type `warranty'.
scale = 20
777897120897129803 / 1740273878919823 + 71893749218 * 102380
7360482044939286.99695278996293989874
71829 / 710238 * 812903
82211.89177008270466486756
```

：计算器交互模式下的复杂表达式运算示例

可以看到结果与bc完全一致。

1.2.3 高精度计算

通过把大数作为字符串解析,计算器能够处理超出常规数据类型精度范围的计算：

```
Error occurred during calculation
wu@w5712:~/CppLearning/CS205/projectFinal1$ ./calculator 7189273879.4891273981 / 481297389123
0.01493728003093705117
wu@w5712:~/CppLearning/CS205/projectFinal1$ echo 7189273879.4891273981 / 481297389123 | bc
0
wu@w5712:~/CppLearning/CS205/projectFinal1$ echo 7189273879.4891273981 / 481297389123 | bc -l
.01493728003093705117
wu@w5712:~/CppLearning/CS205/projectFinal1$
```

：计算器命令行模式下的高精度运算示例(交互模式也可以)

可以看到结果与bc完全一致。

1.3 变量定义与使用

在交互模式中，计算器允许用户通过"[变量名]=[表达式]"的命令来定义变量并在后续计算中使用这些变量。

1.3.1 变量定义与赋值

```
wu@w5712:~/CppLearning/CS205/projectFinal1$ ./calculator
Simple Calculator (type 'quit' to exit)
> a1 = 1008312908
Variable a1 set to 1008312908
> a2 = 1
Variable a2 set to 1
> a1 x a2 x 100 + 1
Result: 100831290801
>
```

：可以定义由字母开头，只包含数字，字母的变量

1.3.2 变量表显示

```
> a3 = a1 x a2
Variable a3 set to 1008312908
> state

=== Current State ===

Variables (3/50):
Name        Value
--------------------
a1          1008312908
a2          1
a3          1008312908

Functions (0/20):
No functions defined

====================
```

：使用state命令显示变量，变量可以用表达式赋值

1.3.3 变量更新

```
====================
> a1 = 1
> state

=== Current State ===

Variables (3/50):
Name        Value
--------------------
a1          1
a2          1
a3          1008312908

Functions (0/20):
No functions defined

====================
>
```

：变量可以重新赋值

1.4 函数定义与调用

计算器支持用户自定义函数并在计算中调用这些函数。 函数的变量与自定义变量共用变量栈,每次调用完后函数的变量会自行释放。

```
Goodbye!
wu@w5712:~/CppLearning/CS205/projectFinal1$ ./calculator
Simple Calculator (type 'quit' to exit)
> _CalculateInCome[Revenue;Expanse;Dividend]{ Revenue - Expanse - Dividend }
Function defined successfully
> /_CalculateInCome[1000;231;12]
Result: 757
> /_CalculateInCome[312491;123;12334
Invalid function call format
> /_CalculateInCome[312491;123;12334]
Result: 300034
>
```

函数的定义与调用

1.4.1 函数定义

通过 函数名[参数表]{表达式} 的方式来定义函数。 函数名称必须以_开头,变量需满足变量的定义,中间以;分割，不能出现任何空格。

1.4.2 函数调用

调用函数时.使用/函数名[参数]的格式,程序会在全局变量表内暂时创建相应的变量,在调用完成后再将变量删除。 函数的参数列表里只能放入数字。

1.5 错误处理机制

我在本次项目中尽力实现了错误处理,并将错误分为了三类 MemoryError,RunError和TokenizeError 第一个对应内存安全,第二个对应诸如除零/重复定义函数/调用未定义变量等问题,第三个则对应输入解析问题。 由于错误种类较多，这里仅列出其中的部分，欢迎使用者自行体验。

1.5.1 语法错误处理

```
Goodbye!
wu@w5712:~/CppLearning/CS205/projectFinal1$ ./calculator  173819278 asd 01asd
TokenizeError: Invalid operator. Use +, -, x, or /
wu@w5712:~/CppLearning/CS205/projectFinal1$ ./calculator adsjkhkhk / hasjkd
TokenizeError: Number can not be interpreted
wu@w5712:~/CppLearning/CS205/projectFinal1$ ./calculator  173819278 asd 01asd
TokenizeError: Invalid operator. Use +, -, x, or /
wu@w5712:~/CppLearning/CS205/projectFinal1$ ./calculator
Simple Calculator (type 'quit' to exit)
> 821908309adad
TokenizeError: The input can not be interpreted as VARIABLE or OPERAND.
> 12390-9-0
TokenizeError: The input can not be interpreted as VARIABLE or OPERAND.
> --
TokenizeError: The input can not be interpreted as VARIABLE or OPERAND.
> _func
TokenizeError: Expected '['
TokenizeError:Invalid function declaration
TokenizeError:Failed to define function
```

语法错误处理

1.5.2 运行时错误处理

```
Goodbye!
wu@w5712:~/CppLearning/CS205/projectFinal1$ ./calculator
Simple Calculator (type 'quit' to exit)
> a
RunError: Variable 'a' not defined
> qeio12937
RunError: Variable 'qeio12937' not defined
> ___f
TokenizeError: Expected '['
TokenizeError:Invalid function declaration
TokenizeError:Failed to define function
> 12331 / 0
RunError: Division by zero
> io1930
RunError: Variable 'io1930' not defined
> quit
Goodbye!
wu@w5712:~/CppLearning/CS205/projectFinal1$ ./calculator 1132.123 / 0
RunError: Division by zero
wu@w5712:~/CppLearning/CS205/projectFinal1$
```

运行时错误处理

1.5.3 内存错误处理

我在这里把变量上限设置为了1

```
wu@w5712:~/CppLearning/CS205/projectFinal1$ ./calculator
Simple Calculator (type 'quit' to exit)
> a = 1
Variable a set to 1
> a1 = 2
MemoryError: Variable table is full
> _func1[a;b]{a + b}
Function defined successfully
> _func1[1;2]
TokenizeError: Invalid variable name: 1
TokenizeError:Invalid function declaration
> /_func1[1;2]
MemoryError: Not enough space in variable table. Need 1 more slots
>
```

内存错误处理

2. 核心难点分析

在这部分中,我将分享一些在项目里遇到的问题和我自己的解决方法。

2.1 高精度计算实现

对于高精度的加法、减法和乘法，我主要采用模拟竖式计算的方法。而对于高精度除法，由于已经有了前面三个大精度算法的基础,我则放弃了通过竖式模拟，而是使用前面的三个方法通过牛顿迭代法进行实现。

高精度加法相关的函数共有三个（减法/乘法的设计完全一致）：add_big_abs、add_big_decimal 以及 complete_add。

这三个方法的适用范围逐步扩展，并形成顺序依赖关系，分别对应正整数加法、正实数加法和实数加法三个类别。

这种模块化设计虽然增加了总代码量，但提升了代码复用性，同时便于层次化测试与功能扩展，有助于更好地定位 bug。我认为这是本次项目中做得较好的一点。

以下主要介绍高精度加法和除法的实现。

2.1.1 高精度加法

2.1.1.1add_big_abs 方法

实现思路：

- 计算两个正整数的长度，确定结果的最大长度。
- 分配内存并初始化结果字符串。
- 从最低位开始模拟竖式加法，逐位相加并处理进位。
- 返回结果字符串。

伪代码：

```
函数 add_big_abs(num1, num2):
    计算 num1 和 num2 的长度 len1 和 len2
    计算结果的最大长度 maxLen
    分配内存给结果字符串 result 并初始化为 '0'
    初始化进位 carry 为 0
    从最低位开始逐位相加:
        计算当前位的和 sum = digit1 + digit2 + carry
        处理进位
        将当前位的结果存入 result
    返回 result
```

2.1.1.2add_big_decimal 方法

实现思路：

- 找到两个数的小数点位置，计算整数部分和小数部分的长度。
- 对齐小数部分，补零使得小数部分长度相同。
- 调用 add_big_abs 方法进行加法计算。
- 在结果中插入小数点，处理前导零。
- 返回最终结果。

伪代码：

```
函数 add_big_decimal(num1, num2):
    找到 num1 和 num2 的小数点位置
    计算整数部分和小数部分的长度
    对齐小数部分，补零
    调用 add_big_abs 进行加法计算，得到 raw_result
    在 raw_result 中插入小数点，处理前导零
    返回 final_result
```

2.1.1.3complete_add 方法

实现思路：

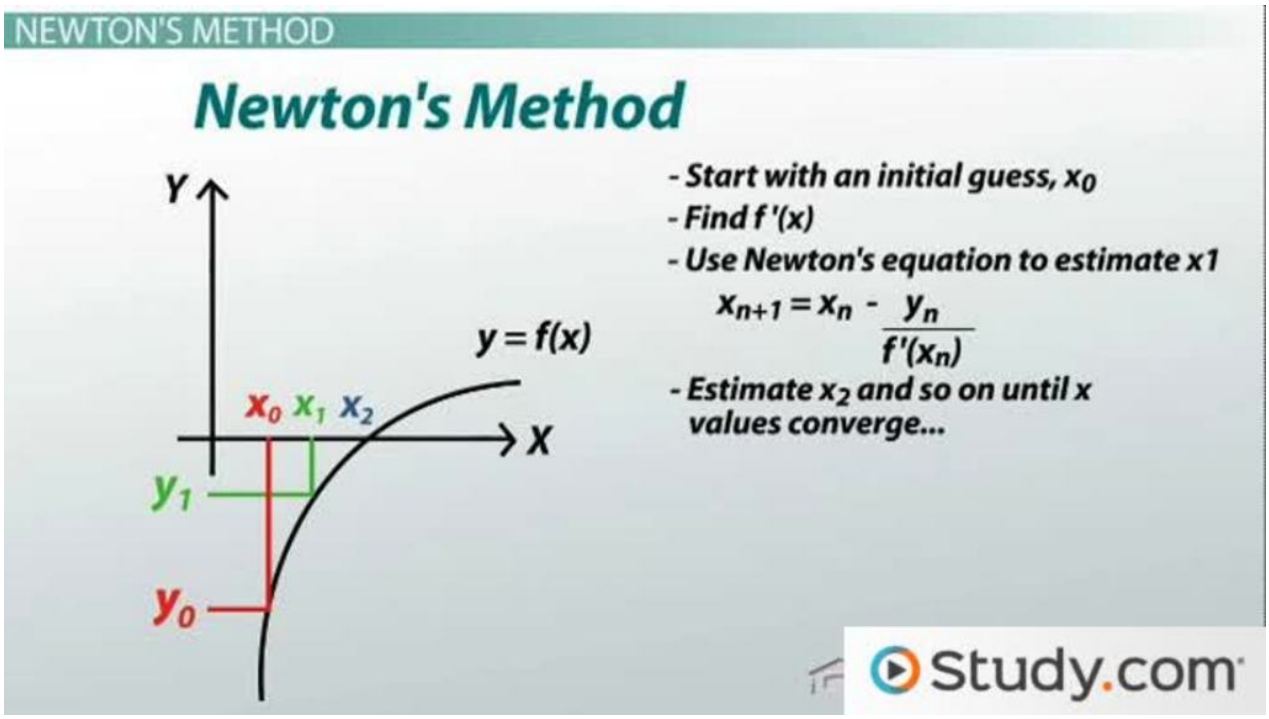
- 检查输入是否带符号，去掉符号部分。
- 根据符号情况决定调用加法或减法方法：
    - 如果两个数都是负数，调用 add_big_decimal 并在结果前加负号。
    - 如果一个数是负数，另一个数是正数，调用 subtract_big_decimal 进行减法运算。

    - 如果两个数都是正数，直接调用 add_big_decimal。
- 返回结果。

（思路很简单就不放伪代码了）

2.1.2 高精度除法

由于之前已经实现大数的加减乘,通过牛顿迭代公式来逼近需要的除法精度是比较合适的。



牛顿迭代法公式

这种方法的实现原理并不困难，我们只需要将f(x) = 1/x及其导数代入公式，就可以得到通过数次迭代来得到指定精度的倒数值的公式为x_n+1 = x_n * (2 - d * x_n)。

以下是牛顿迭代法实现高精度除法的伪代码：

```
函数 newton_divide(dividend, divisor, precision):
    初始化 x0 为 1 / divisor 的近似值
    设置迭代次数 max_iterations
    设置误差阈值 epsilon

    对于 i 从 1 到 max_iterations:
        计算 x1 = x0 * (2 - divisor * x0)
        如果 |x1 - x0| < epsilon:
            跳出循环
        更新 x0 = x1

    计算结果 result = dividend * x0
    返回 result
```

虽然伪代码看起来十分简单,实现的代码长度也不多,但是我却在这里被卡了很久。

为什么呢？

首先是因为内存管理问题。由于最开始设置精度过大，牛顿迭代循环会进行非常多次，我原来就存在的内存泄漏问题终于被察觉。

好在，由于之前一直又在尝试注意内存管理问题，并且有copilot的随时提醒，我的内存问题并没有想象中的那么严重，在解决掉一些变量的double free问题以及部分变量没有及时free的问题之后，根据valgrind的显示，我的内存泄漏问题（至少是在除法上）得到了解决。

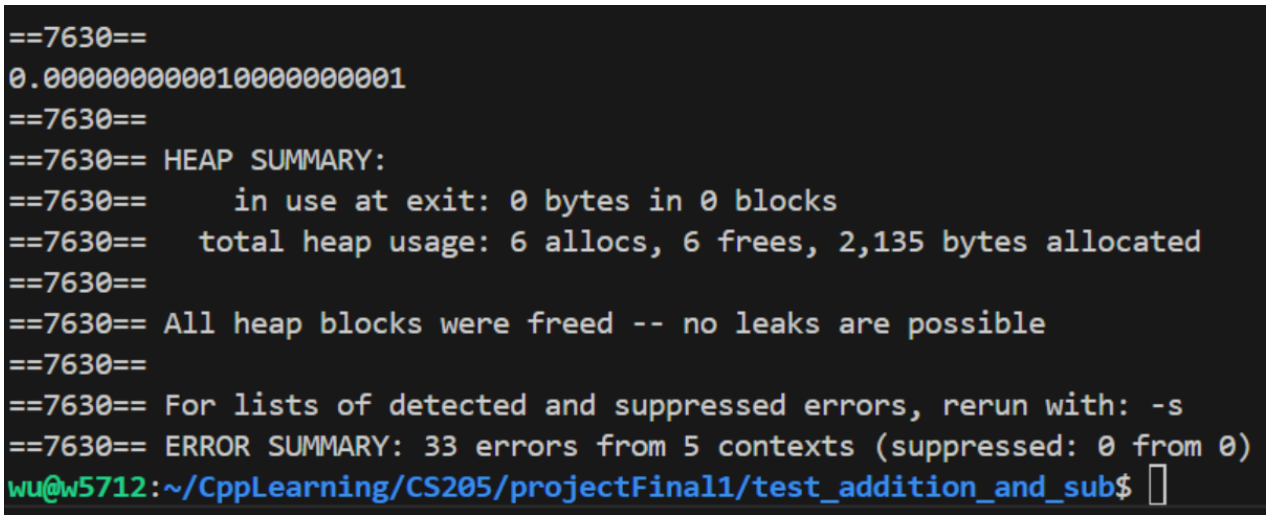
然而在此之后，我的牛顿迭代却仍然无法正常工作。我本来以为是精度和初始值的选择问题，然而在参照Monad学长的报告设置初始值和精度后，我的问题仍没有得到解决。

就在我一筹莫展的时候，通过valgrind的溯源功能，我才发现原来问题是处在了之前我实现的大数乘法...

bug并不是非常的难修,主要问题出现在我的乘法在数据过小时会出现小数点位置错误的问题。

然而，这次痛苦的debug的经历也给了我留下了极深的印象。**也是在这次bug出现之后,我对于所有的基础模块都进行了尽可能完善测试，使用bc计算机器来与我的计算模块进行对拍并使用脚本检测结果。**

不过也正是为了对这一部分debug,我安装并且简要地学习了valgrind工具的使用。它提供的报告很大程度上提升了我排除bug的效率（尤其是字符串的相关操作！）也可以说是收获颇丰。



修bug过程中留下的一张截图，当时十分绝望

2.2 表达式解析

在还没有开始实施项目的时候，我对于整个项目架构的构想其实与bc计算器类似(可以参见摘要中的图片)，我也希望把变量和数值来作为基本元素（没有注释因为不准备支持脚本功能），并在此之上通过方程来简化一些操作。

然而在实际的开发中，由于时间限制和解析相关的bug过多等等因素，我放弃了把方程纳入表达式树的可处理范围（即途中从方程指向表达式树的箭头），而是仅仅允许把方程的内部解析为单行表达式。

不过即便如此，我认为可以算是本项目亮点之一的是，我仍然形成了一套以表达式树解析和计算为核心的模块化的计算体系。

不管是变量还是数据还是操作符，它们在计算过程中都会在表达式树的构建函数 tokenize函数中被转化，其中变量和数据为叶节点，操作符则为根节点。

这样的操作减少了变量的特殊性，使得我的代码模块化程度提高了许多。

正如图片中所示，我的表达式解析部分的主要函数有两个，其一是将中缀表达式转化为树的tokenize方法，其二则是递归对数进行计算的calculate_core方法，接下来我将简要的讲解这两个方法的实现思路。

正如图片中所示，我的表达式解析部分的主要函数有两个，其一是将中缀表达式转化为树的tokenize方法，其二是递归对数进行计算的 calculate_core方法。接下来，我将简要讲解这两个方法的实现思路。

2.2.1 tokenize方法

该方法的目的是将中缀表达式转化为表达式树。它使用两个栈，一个用于操作符 (operatorStack)，一个用于操作数 (operandStack)。通过遍历输入字符串，将操作符和操作数分别压入相应的栈中，并根据操作符的优先级构建表达式树。

算法流程：

```
函数 tokenize(input):
    创建操作符栈 operatorStack
    创建操作数栈 operandStack
    创建输入字符串的副本 inputCopy
```

```
    将 inputCopy 按空格分割为 token

    对于每个 token:
        如果 token 是操作符:
            当 operatorStack 不为空 且 栈顶操作符优先级 >= 当前操作符:
                弹出 operatorStack 栈顶操作符
                弹出 operandStack 栈顶两个操作数
                创建新节点，左子树为第一个操作数，右子树为第二个操作数
                将新节点压入 operandStack
            将当前操作符压入 operatorStack

        如果 token 是左括号:
            将左括号压入 operatorStack

        如果 token 是右括号:
            当 operatorStack 栈顶不是左括号:
                弹出 operatorStack 栈顶操作符
                弹出 operandStack 栈顶两个操作数
                创建新节点，左子树为第一个操作数，右子树为第二个操作数
                将新节点压入 operandStack
            弹出 operatorStack 栈顶左括号

        如果 token 是操作数:
            创建新节点，值为 token，类型为操作数
            将新节点压入 operandStack

    当 operatorStack 不为空:
        弹出 operatorStack 栈顶操作符
        弹出 operandStack 栈顶两个操作数
        创建新节点，左子树为第一个操作数，右子树为第二个操作数
        将新节点压入 operandStack

    返回 operandStack 栈顶节点（即表达式树的根节点）
```

2.2.2 calculate_core方法

calculate_core方法的目的是递归计算表达式树的值。它通过递归遍历表达式树的节点，根据操作符对左右子树的值进行相应的运算，并返回计算结果。

递归计算流程：

```
函数 calculate_core(root):
    如果 root 为空:
        返回 NULL

    如果 root 是叶子节点:
        如果 root 类型是变量:
            查找变量值并返回
        否则:
            返回 root 的值

    递归计算左子树的值 leftValue
    递归计算右子树的值 rightValue

    如果 leftValue 或 rightValue 为空:
        释放 leftValue 和 rightValue
        返回 NULL

    根据 root 的操作符进行相应的运算:
        如果操作符是 '+':
            result = complete_add(leftValue, rightValue)
        如果操作符是 '-':
            result = complete_sub(leftValue, rightValue)
        如果操作符是 'x':
            result = complete_mul(leftValue, rightValue)
        如果操作符是 '/':
            如果 rightValue 是 "0":
                打印错误信息
```

```
                释放 leftValue 和 rightValue
                返回 NULL
            result = complete_div(leftValue, rightValue, scale)

    释放 leftValue 和 rightValue
    返回 result
```

由于这两个方法的存在,我的数据处理函数(加减乘除)并不会直接地接处用户输入

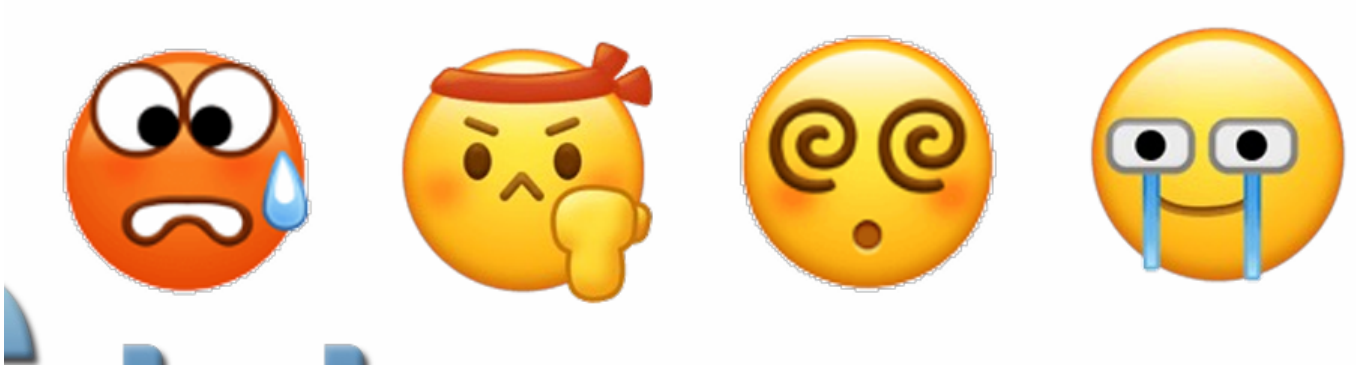
这使得我的代码的健壮程度得到了增加,也使得我的计算函数省略检查数据合法性的部分(在转化为表达式树的时候就已经实现)。

除了以上所说的两个难点之外，其实诸如字符串处理，内存管理等等问题也十分得困扰我。但是由于我觉得自己的实现没有太多的可圈可点之处，甚至可能在部分地方还有问题，我决定不单独对他们进行分点阐述，并直接进入项目总结部分。

3. 项目完成感言

在大约30多个小时的艰难开发之后，我终于完成了自己人生中的第一个特别特别简单的C语言项目。

在项目的开发流程里，我觉得引用这张图来表达我的心情会比较合适：



最开始，于老师要求我们使用AI进行开发。于是，我花费了大约5分钟（感觉是因为网络卡顿）就完成了整个项目。

这就结束了吗？我感到焦虑和茫然。由于完全没有C语言基础，我连AI写的代码其实都看不懂。那该怎么办呢？

我还是决定放弃了AI给出的架构，并且通过浏览bc文档和看之前学长的报告的方式来设计自己的项目，逐渐对于整个开发流程有了一个掌握。

在此之后，由于感觉直接使用gmp等高精度运算库会学到的很少，我决定手动实现大数的四则运算；又因为机组正在学习的caller和callee相关的知识以及上学期dsaa所讲的表达式树，我设计出了一个初始的项目结构...随着思路的开拓,我的需求也逐渐明朗。也逐渐变成了图片里第二张表情的状态。

之后就进入了开发阶段。很不幸，由于缺乏项目和这门语言的经验，我感觉我此时的状态就像是第三幅表情，非常的晕，十分被动，遇到bug也可以改出来，但是感觉对自己的代码能力信心严重不足，每每写一个新的方法都会往回看担心自己之前的方法有bug。

到最后，也是稀里糊涂地改好了所有能找到的bug，写完报告，放弃改不好的功能并且伤心地提交代码。

以上就是我这次项目的大致流程。

不过，虽然我感觉自己这次的项目写的不是很完美，我还是花费了巨量的时间来尝试，从中学到了很多知识并且锻炼了自己的心态。

例如C语言的字符串处理相关方法，valgrind的使用以及基础内存管理方法，linux系统的使用，bc的操作，指针的使用（这个可能没会）等等等等。

不过项目也有大量的遗憾，除了之前说的没把方程纳入表达式体系之外，我的代码结构（由于同学在群里的发言把超级多的文件整合在了一起，形成了奇怪的代码结构），算法复杂度（karatsuba算法，其实已经实现但是由于没时间测试bug所以我还是使用了竖式相乘），错误处理（整合成一个方法来统一处理错误），内存管理（使用链表管理内存？）等等等等。我希望也可以再下一次项目中进行改进。

以上就是我简要的个人感想，希望可以慢慢的学会C和CPP。

2025/3/15

参考文献

1. bc手册
2. HaiBinLai学长的仓库
3. 一个简单但是很完善的C语言计算器
4. Monad学长的报告